adapty | Docs

INTEGRATION GUIDE

Display flows in iOS UIKit apps

Minimal setup to fetch and present an Adapty flow in a UIKit app — three steps and the required delegate methods. Works with both Flow Builder and Paywall Builder placements.

UIKit | iOS SDK v4 (beta) | Flow Builder + Paywall Builder

PLATFORM	SDK VERSION	DOCUMENT
iOS · UIKit	v4.0 (beta)	Integration Guide

This guide shows the minimum code needed to fetch a flow from Adapty and present it in a UIKit app. The same three steps work for placements created in the Flow Builder or in the classic Paywall Builder — no dashboard changes required.

CONTENTS

1. Get the flow ›
2. Get the flow configuration ›
3. Present the flow ›
Full example ›

1. Get the flow

Fetch the flow object by placement ID. The flow is the placement's content for the current user — products, A/B test variant, and the view configuration link.

```swift
import Adapty

let flow = try await Adapty.getFlow(placementId: "YOUR_PLACEMENT_ID")
```

Optional parameters:

Parameter	Default	What it affects
`fetchPolicy`	`.reloadRevalidatingCacheData`	Cache strategy. `.returnCacheDataElseLoad` returns cache when present; `.returnCacheDataIfNotExpiredElseLoad(maxAge: )` returns cache only if newer than `maxAge` seconds.
`loadTimeout`	`5s`	Maximum time to wait for a network response before falling back to cached data or throwing.

2. Get the flow configuration

Fetch the visual configuration — the assets and layout the SDK needs to render the flow on screen.

```swift
import AdaptyUI

let flowConfiguration = try await AdaptyUI.getFlowConfiguration(forFlow: flow)
```

Optional parameters:

Parameter	Default	What it affects
`loadTimeout`	`5.0`	Maximum time to wait for the view configuration to load. Minimum 1.0 second.
`products`	`nil`	Pre-loaded `[AdaptyPaywallProduct]` to skip the SDK's internal product fetch. Pass the result of `Adapty.getPaywallProducts(flow:)` if you have already loaded them.
`observerModeResolver`	`nil`	Required only when the SDK is initialized in observer mode. Bridges Adapty purchase events to your existing purchase code.
`tagResolver`	`nil`	Closure that returns string values for custom `{tag}` placeholders defined in the paywall text.
`timerResolver`	`nil`	Closure that returns end dates for personalized countdown timers configured in the paywall.
`systemRequestsHandler`	`nil`	Hook for the SDK's system-level network requests. Advanced — leave `nil` for normal use.

3. Present the flow

Build the controller and present it like any other `UIViewController`. The factory needs the configuration and a delegate.

```swift
import AdaptyUI

let controller = try AdaptyUI.flowController(
    with: flowConfiguration,
    delegate: self
```

```
)

present(controller, animated: true)
```

Optional parameters:

Parameter	Default	What it affects
`showDebugOverlay`	`false`	When `true`, the controller draws a debug overlay with placement and variant info. Use only during development.

The delegate, `AdaptyFlowControllerDelegate`, has default implementations for almost every method. The three you **must** provide handle the three failure paths the SDK can't decide for you:

```swift
extension YourViewController: AdaptyFlowControllerDelegate {
    func flowController(_ controller: AdaptyFlowController,
                        didFailPurchase product: AdaptyPaywallProduct,
                        error: AdaptyError) {
        // Show an error to the user.
    }

    func flowController(_ controller: AdaptyFlowController,
                        didFinishRestoreWith profile: AdaptyProfile) {
        // Check profile.accessLevels and dismiss if access is granted.
    }

    func flowController(_ controller: AdaptyFlowController,
                        didFailRestoreWith error: AdaptyError) {
        // Show an error to the user.
    }
}
```

The remaining delegate methods are optional. Implement them only when you need to override the default behavior or hook in your own logic.

Method	Default behavior	When to override
`didAppear`	No-op.	Track an analytics event when the flow appears.
`didDisappear`	No-op.	Track an analytics event when the flow is dismissed.

Method	Default behavior	When to override
`didPerform action:`	`.close` dismisses the controller; `.openURL` opens the URL in Safari; `.custom` is a no-op.	Handle a custom action button, or replace the close or open URL behavior.
`didSelectProduct`	No-op.	React when the user taps a product (for example, to log a custom event).
`didStartPurchase`	No-op.	Show a loading indicator while the purchase is in flight.
`didFinishPurchase`	Dismisses the controller on successful purchase.	Run your own post-purchase logic (analytics, unlock content) before or instead of dismissing.
`didStartRestore`	No-op.	Show a loading indicator during restore.
`didFailRenderingWith`	No-op.	Dismiss the controller and surface an error to the user.
`didFailLoadingProductsWith`	Returns `false` (do not retry).	Return `true` to ask the SDK to retry the products fetch.
`didPartiallyLoadProducts`	No-op.	Log which product IDs failed to load.
`didFinishWebPaymentNavigation`	No-op.	Handle the result of a web-based payment flow.
`didReceiveAnalyticEvent`	No-op.	Forward flow-level analytics events to your own provider (Amplitude, Mixpanel, etc.).

Full example

SWIFT

```swift
import Adapty
import AdaptyUI
import UIKit

final class PaywallPresenter: UIViewController {

    func presentPaywall() {
        Task { @MainActor in
            do {
                let flow = try await Adapty.getFlow(placementId: "YOUR_PLACEMENT_ID")
                let configuration = try await AdaptyUI.getFlowConfiguration(forFlow: flow)

                let controller = try AdaptyUI.flowController(
                    with: configuration,
                    delegate: self
                )
                present(controller, animated: true)
            } catch {
                print("Failed to present paywall: \(error)")
            }
        }
    }
}

extension PaywallPresenter: AdaptyFlowControllerDelegate {

    func flowController(_ controller: AdaptyFlowController,
                        didFailPurchase product: AdaptyPaywallProduct,
                        error: AdaptyError) {
        print("Purchase failed: \(error)")
    }

    func flowController(_ controller: AdaptyFlowController,
                        didFinishRestoreWith profile: AdaptyProfile) {
        print("Restore finished — check access levels and dismiss if granted.")
    }

    func flowController(_ controller: AdaptyFlowController,
                        didFailRestoreWith error: AdaptyError) {
        print("Restore failed: \(error)")
    }
}
```